[LOCAL TEST] Heuristic Approaches for the k-TSP

Barrientos Gatica

Abstract

Local compilation test of the OPSEARCH paper body.

1 Introduction

The k-Traveling Salesman Problem (k-TSP) is a generalization of the classical Traveling Salesman Problem (TSP) that has received growing attention in the combinatorial optimization literature (Venkatesh et al., 2018; Pandiri and Singh, 2020; Ene et al., 2017; Blanchard et al., 2023). Unlike the standard TSP, in which a single salesman must visit *all* the cities, in the k-TSP the salesman visits only a *subset* of exactly k cities (including a fixed home city) and the remaining $n - k$ cities are left unvisited. The objective is to minimize the total length of the single resulting closed tour.

Formally, given n cities with a designated home city (depot) and a fixed value k with $1 < k \leq n$, a feasible solution is obtained by first *selecting* which k cities to include (the home city among them) and then determining a *circular ordering* of those k cities (Venkatesh et al., 2018). This dual (selection + ordering) nature distinguishes the k-TSP from the standard TSP and is the source of its combinatorial difficulty: choosing a subset of k nodes among n and then ordering them yields a search space that grows combinatorially, making exhaustive enumeration intractable for medium and large instances (Lawler et al., 1985). The TSP is recovered as the special case $k = n$, so the k-TSP is NP-hard (Rego et al., 2011).

The k-TSP also sits within the broader family of *TSP with profits* and selective routing problems (Feillet et al., 2005). In particular, it is the unit-prize special case of the Quota TSP, itself a variant of the Prize-Collecting TSP introduced by Balas (1989): when every node carries a unit prize and the quota equals k, collecting the quota is exactly visiting k nodes (Bienstock et al., 1993). A recent systematic survey of TSP variants places these selective problems in a common definitional framework (Hougardy and Zhong, 2025).

Beyond the selective-routing framing, a large body of work targets the classical TSP itself. Exact methods based on integer and dynamic programming solve it optimally but scale poorly with n (Dantzig et al., 1954; Bellman, 1962; Held and Karp, 1962; Karp, 1972), which motivated decades of research on construction heuristics and local search (Lin, 1965; Christofides, 1976; Bentley, 1992; Johnson and McGeoch, 1997; Laporte, 1992). Representative metaheuristic families include tabu search (Glover, 1986; Glover and Laguna, 1997; Knox, 1994), simulated annealing (Černý, 1985), ant colony optimization (Dorigo and Gambardella, 1997), variable neighborhood search (Mladenović and Hansen, 1997) and iterated local search (Martin et al., 1991), as well as evolutionary approaches for the TSP (Potvin, 1996); broad overviews and design principles are given in (Blum and Roli, 2003; Talbi, 2009). Comprehensive treatments of the TSP and its variants (Gutin and Punnen, 2007), near-optimal exact solvers (Applegate et al., 2006) and polynomial-time approximation schemes for the Euclidean case (Arora, 1998) complete this landscape, within which node-selection variants such as the multiple TSP (Bektaş, 2006) and the k-TSP naturally arise.

For the k-TSP specifically, Venkatesh et al. (2018) proposed a general variable neighborhood search (GVNS) heuristic, and Pandiri and Singh (2020), in this journal, introduced the first meta-heuristic and hyper-heuristic approaches (GVNS and two hyper-heuristics, HH_RAND and HH_GREEDY) together with a benchmark of 76 TSPLIB instances that has since become the reference testbed. The present study adopts exactly that benchmark and experimental protocol, so that our results are directly comparable with those of Pandiri and Singh (2020).

The main motivation for studying the k-TSP lies in applications where only a subset of locations must be served under a budget on the number of stops: selective sampling, prize-collecting routing, surveillance with limited visits, and resource-constrained logistics (Vansteenwegen et al., 2011; Blanchard et al., 2023).

Contribution. This work contributes (i) a reproducible, open implementation that models and solves the k-TSP with *ten* methods under a common local-search core, namely a greedy cheapest-insertion heuristic, a GRASP/multi-start scheme, a genetic algorithm (GA), the classical Nearest Neighbor (NN) and Nearest, Farthest and Random Insertion heuristics (NI, FI, RI), Simulated Annealing (SA), Tabu Search (TS), and a proposed Deterministic Cheapest-Insertion Heuristic (DCIH); (ii) an evaluation on the 76-instance benchmark of Pandiri and Singh (2020) under their short- and long-run time protocol, using the Lin–Kernighan–Helsgaun solver (LKH-3) as the best-known-solution reference; and (iii) a statistical comparison, via the two-tailed Wilcoxon signed-rank test, against the published GVNS and hyper-heuristic results. The remainder of the paper presents the problem and model (Section 2), the solution methods (Section 3), the experimental setup (Section 4), the results and discussion (Section 5) and the conclusions (Section 6).

2 Problem definition and mathematical model

The k-TSP must choose *which* k nodes to visit and *in which order*. The number of candidate subsets is $\binom{n-1}{k-1}$ and each admits $(k-1)!/2$ distinct tours; for $n = 100$ and $k = 50$ the number of subsets alone exceeds 10^{28}, so heuristic and metaheuristic methods are required.

Let $G = (V, E)$ with $V = \{0, 1, \ldots, n-1\}$ and node 0 the home city (depot), and let c_{ij} be the distance between nodes i and j. We use a single-salesman selective formulation with two families of binary variables: $y_i = 1$ if node i is *selected* (visited) and 0 otherwise; and $x_{ij} = 1$ if the tour traverses arc (i, j) and 0 otherwise. Ordering variables u_i (Miller–Tucker–Zemlin, Miller et al., 1960) eliminate subtours among the selected nodes. The model minimizes the total tour length

$$\min \; Z = \sum_{i \in V} \sum_{\substack{j \in V \\ j \neq i}} c_{ij}\, x_{ij}, \tag{1}$$

subject to the selection cardinality (exactly k nodes, home city included)

$$\sum_{i \in V} y_i = k, \qquad y_0 = 1, \tag{2}$$

the degree constraints linking arcs to selection

$$\sum_{\substack{j \in V \\ j \neq i}} x_{ij} = y_i, \qquad \sum_{\substack{j \in V \\ j \neq i}} x_{ji} = y_i, \qquad \forall\, i \in V, \tag{3}$$

and the MTZ subtour-elimination constraints over the selected nodes

$$u_0 = 0, \quad 1 \leq u_i \leq k-1 \;\; \forall i \neq 0, \quad u_i - u_j + (k-1)\, x_{ij} \leq k-2 \;\; \forall\, i, j \neq 0,\; i \neq j, \tag{4}$$

with $x_{ij}, y_i \in \{0, 1\}$ and $u_i \geq 0$. This is a selective single-tour TSP: it differs from the standard TSP by the selection variables y_i and the cardinality constraint $\sum_i y_i = k$, and from the multiple-TSP by having a single salesman. Related selective routing models (orienteering, prize-collecting) share the node-selection mechanism (Vansteenwegen et al., 2011; Gunawan et al., 2016); the k-TSP fixes the number of visits to exactly k and minimizes length, as defined by Venkatesh et al. (2018).

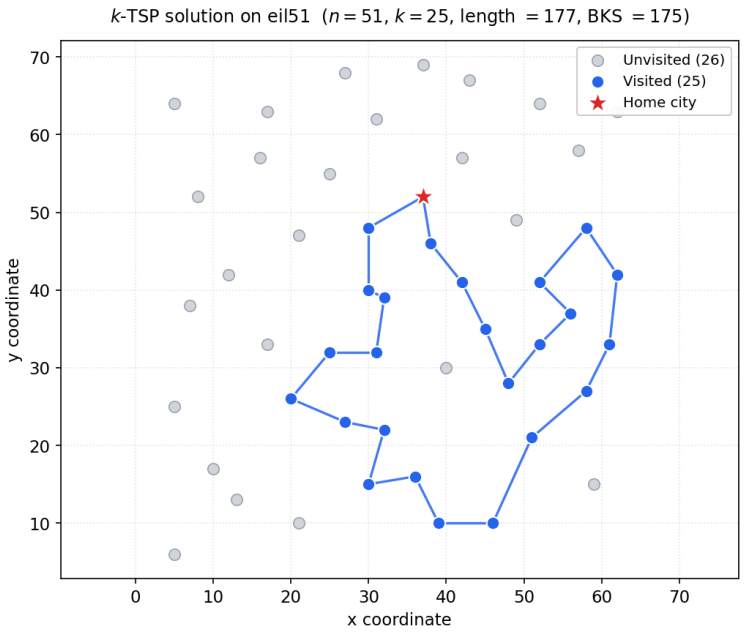


Figure 1: A k-TSP solution on `eil51` with $k = 25$ ($\lfloor n/2 \rfloor$). A single tour (blue) through the home city (red star) visits the k selected nodes; the remaining $n - k$ nodes (gray) are left unvisited

3 Solution methods

All methods share a common representation and local-search core, which makes the comparison fair. A solution is encoded by the selected subset of k nodes and their cyclic order (a genotype that decodes into a single closed tour through the home city, Fig. 2). The common refinement combines a 2-opt move over the cycle (Croes, 1958) with an *in/out exchange* that replaces a selected node by an unselected one, so that both the ordering and the selection are improved.

Greedy (cheapest insertion). Starting from the home city, the tour grows by repeatedly inserting the node with the smallest marginal insertion cost $\Delta(v, u \to w) = d(u, v) + d(v, w) - d(u, w)$ at its best position, until it contains k nodes; it is then refined by the common local search. Several deterministic restarts (different second nodes) keep the best tour (Rosenkrantz et al., 1977; Gutin et al., 2002).

GRASP / multi-start. Following the GRASP framework (Feo and Resende, 1995), a randomized greedy construction picks, with controlled noise, nodes close to the home city, builds a tour and applies local search; the best tour over many restarts within the time budget is kept.

Genetic Algorithm (GA). Each individual encodes the subset and its order (Holland, 1975). The GA uses binary-tournament selection, a subset-aware crossover that repairs cardinality, a mutation that swaps positions and exchanges in-/out-of-tour nodes, elitism and optional 2-opt refinement (Potvin, 1996). Because crossover and mutation change *which* nodes are selected, the GA explores the selection space as well as the ordering space.

Nearest Neighbor (NN). The classic constructive heuristic (Rosenkrantz et al., 1977): from the home city the tour moves to the nearest not-yet-visited node until k nodes are selected, then is refined. Ties are broken by node index, so NN is deterministic.

Nearest and Farthest Insertion (NI, FI). Two members of the insertion family (Rosenkrantz et al., 1977; Bentley, 1992). Both place the incoming node at its cheapest position; they differ in

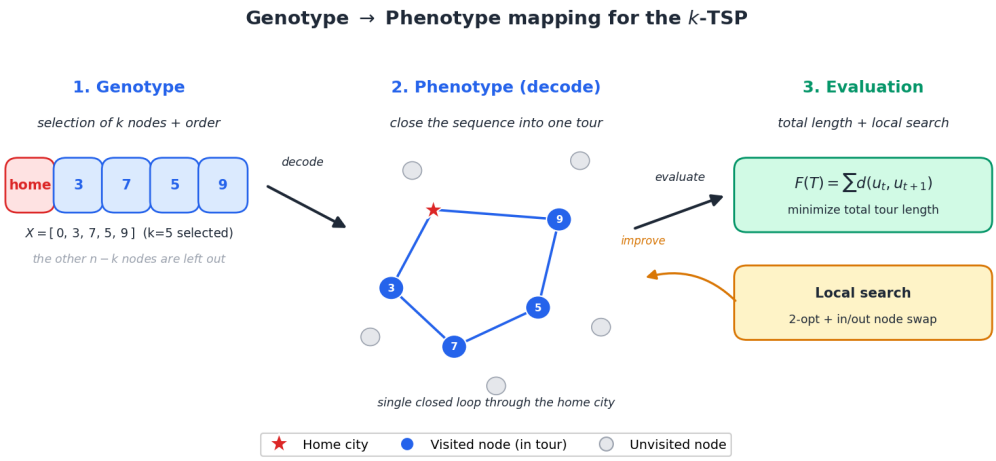


Figure 2: Genotype→phenotype flow: the genotype encodes the selected subset of k nodes and their order; decoding closes the sequence into a single tour through the home city, evaluated by its total length and refined by local search

which outside node is inserted next — the one *closest* to the tour (NI) or *farthest* from it (FI). Both are deterministic.

Random Insertion (RI). A stochastic multi-start insertion variant (Rosenkrantz et al., 1977): the node to insert is chosen uniformly at random and placed at its cheapest position; the best tour over many restarts is kept.

Simulated Annealing (SA). A single-solution metaheuristic (Kirkpatrick et al., 1983; Černý, 1985). From a Nearest-Insertion tour, SA explores the 2-opt and in/out-exchange neighbourhoods, accepting a move with cost change Δ always if $\Delta < 0$ and otherwise with probability $\exp(-\Delta/T)$, with T cooled geometrically.

Tabu Search (TS). A memory-based metaheuristic (Glover, 1986; Glover and Laguna, 1997; Knox, 1994). From a Nearest-Insertion tour, at each iteration TS applies the best non-tabu move in the same two neighbourhoods; the attributes of applied moves are stored in a tabu list of fixed tenure, with an aspiration criterion that overrides the tabu status when a move yields a new global best.

DCIH (proposed). The Deterministic Cheapest-Insertion Heuristic (Algorithm **??**) builds the tour by cheapest insertion with ties broken by node index (Phase 1) and then refines it deterministically (Phase 2). Given the same instance it always returns the same tour, with no random seed, which makes it fully reproducible.

[Algorithm 1: DCIH — omitted in local test, uses algorithm2e]

As a near-optimal reference we use the Lin–Kernighan–Helsgaun solver (Helsgaun, 2000; y B. W. Kernighan, 1973), whose extension LKH-3 (Helsgaun, 2017) supports the k-TSP natively; the best-known solutions (BKS) are those obtained by LKH-3 on the benchmark of Pandiri and Singh (2020). LKH-3 is used only as the reference and is not among the methods competing under the time budget.

4 Experimental setup

To make our results directly comparable with the state of the art, we follow the exact experimental protocol of Pandiri and Singh (2020). The benchmark comprises the **76 TSPLIB instances**

Table 1: Mean relative gap (%) to the LKH-3 best-known solution, averaged over the 76 TSPLIB instances, for each termination criterion (SR = $0.05n$ s, LR = $0.2n$ s) and k scenario. Methods marked with * are the published results of Pandiri and Singh (2020). Lower is better; the best value in each column is in bold

	Short run (SR)			Long run (LR)		
Method	Small	Medium	Large	Small	Medium	Large
Greedy	4.73	**9.11**	11.59	4.46	8.43	10.41
GRASP	14.26	14.72	13.34	13.12	13.80	12.43
GA	18.46	14.42	11.05	13.82	10.49	8.70
NN	19.48	20.54	15.46	19.48	20.54	15.46
NI	16.82	19.38	18.78	16.82	19.38	18.78
FI	102.34	57.52	34.05	102.34	57.52	34.05
RI	42.57	23.26	13.47	38.48	21.94	12.25
SA	10.06	12.80	13.19	9.79	12.42	12.86
TS	7.04	10.13	**11.00**	6.23	9.08	9.42
DCIH	15.45	20.11	17.06	15.45	20.11	17.06
GVNS*	12.97	24.96	23.03	6.51	14.07	15.71
HH_RAND*	**3.09**	9.56	11.55	**2.42**	**4.50**	**6.22**
HH_GREEDY*	3.87	11.29	13.21	2.97	5.22	7.07

(Reinelt, 1991) of that study, with sizes from $n = 14$ to $n = 783$. For each instance, three k scenarios are evaluated: *Small* $k = \lfloor n/4 \rfloor$, *Medium* $k = \lfloor n/2 \rfloor$ and *Large* $k = \lfloor 3n/4 \rfloor$, giving $76 \times 3 = 228$ test cases. Two wall-clock termination criteria are used: a **short run** (SR) of $0.05\,n$ seconds and a **long run** (LR) of $0.2\,n$ seconds. Every stochastic method is executed **ten** independent times per case and the best tour is reported; the deterministic constructions (NN, NI, FI, DCIH) are run once, as repeated runs would return the identical tour.

Solution quality is measured by the relative gap to the LKH-3 best-known solution, $\text{gap}\% = 100\,(\text{length} - \text{BKS})/\text{BKS}$, where lower is better and a gap of 0 means the method matches LKH-3. For the statistical comparison between algorithms we use the two-tailed **Wilcoxon signed-rank test** (Knox, 1994) on the paired per-instance gaps, the non-parametric paired test recommended for this setting. All methods were implemented in Python and share the same distance routines and local-search core; the implementation and scripts needed to regenerate every table and figure are released in the project repository.

5 Results and discussion

Table 1 reports the mean relative gap to LKH-3 over the 76 instances for every method, termination criterion and k scenario, alongside the published results of Pandiri and Singh (2020) (marked with *). Fig. 3 visualizes the same aggregates for the strongest methods, and Fig. 4 shows the distribution of per-instance gaps under the long run.

Overall ranking. Averaged over all six groups, the hyper-heuristics of Pandiri and Singh (2020) remain the strongest (HH_RAND 6.2%, HH_GREEDY 7.3%), immediately followed by our *Greedy* cheapest-insertion heuristic (8.1%) and *Tabu Search* (8.8%). Both of these clearly outperform the GVNS of the same study (16.2%). Simulated Annealing (11.8%), the genetic algorithm (12.8%) and GRASP (13.6%) form a middle group, while DCIH (17.5%) provides a fully deterministic alternative of quality comparable to the insertion constructions. Random Insertion and, especially, Farthest Insertion are the weakest: inserting the farthest node is counter-productive for the k-TSP, where cheap-to-include nodes should be preferred.

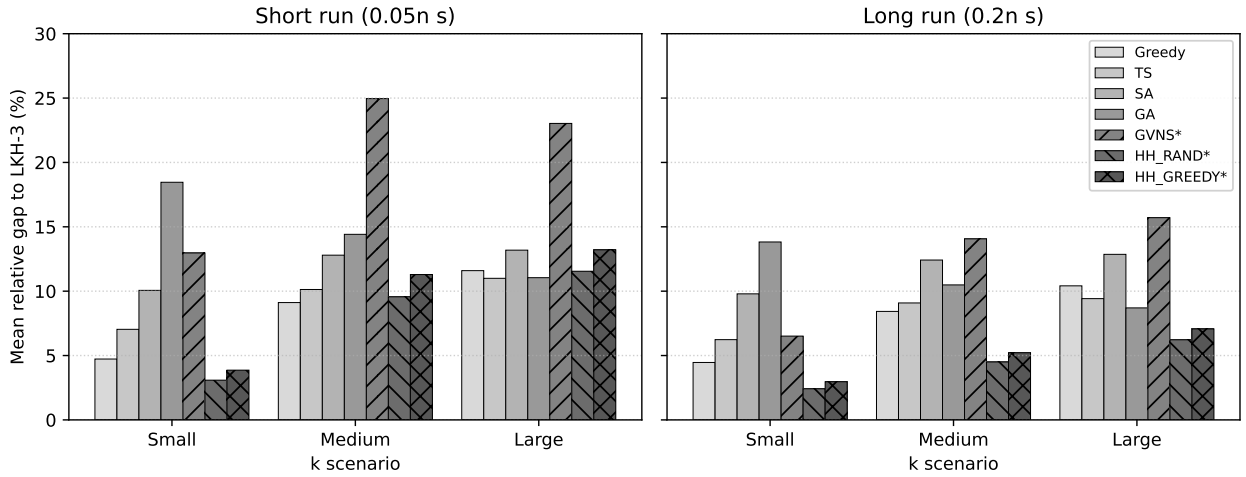


Figure 3: Mean relative gap to LKH-3 (%) by k scenario, for the short run (left) and long run (right). Our methods (solid bars) are shown next to the published GVNS and hyper-heuristics of Pandiri and Singh (hatched bars, *)

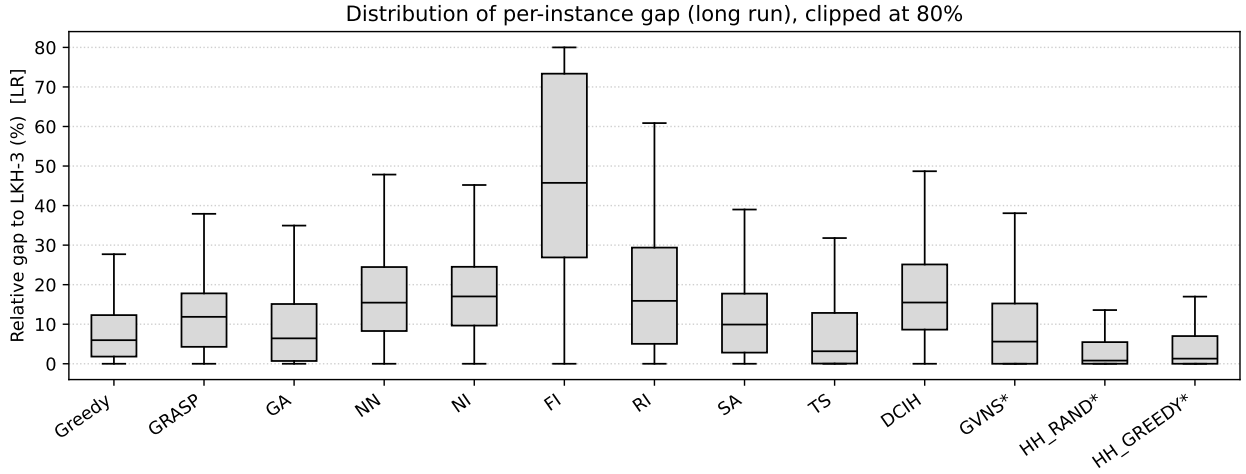


Figure 4: Distribution of the per-instance relative gap to LKH-3 under the long run, aggregated over the three k scenarios (whiskers without outliers; values clipped at 80% for readability). Methods marked * are those of Pandiri and Singh

Effect of the time budget. The deterministic constructions do not change between SR and LR, whereas the time-driven methods improve under the long run. The genetic algorithm benefits the most in the *Large* scenario, where its mean gap falls from 11.0% (SR) to 8.7% (LR), becoming the best of our methods there. Greedy and TS are the most robust across scenarios and budgets, staying near or below 10% in most groups.

Comparison with the state of the art. Table 2 reports the Wilcoxon signed-rank test between our two strongest methods and the three approaches of Pandiri and Singh (2020). Both Greedy and TS are *significantly better than GVNS* under both termination criteria ($p < 0.05$). Under the short run, our Greedy heuristic is even significantly better than HH_GREEDY, and TS is statistically on par with it; the randomized hyper-heuristic HH_RAND remains ahead. Under the long run, with more time available, the hyper-heuristics pull clearly ahead of our single-core methods. Counting per-case wins and ties, the best of our ten methods matches or beats the best of the three Pandiri methods in 103 of the 228 short-run cases (45%) and, against HH_GREEDY alone, in 123 of 228 cases (54%). These results show that simple, reproducible heuristics built on a common local-search

Table 2: Two-tailed Wilcoxon signed-rank test (paired on the per-instance relative gap over the 76 instances) between our two strongest methods and the approaches of Pandiri and Singh (2020). A negative Δ means our method has a smaller (better) mean gap; $p < 0.05$ denotes a significant difference

Crit.	Comparison	Δ gap (%)	p-value	Better
SR	Greedy vs GVNS	-11.84	0.011	Greedy
SR	Greedy vs HH_RAND	+0.41	<0.001	HH_RAND
SR	Greedy vs HH_GREEDY	-0.98	<0.001	Greedy
SR	TS vs GVNS	-10.93	<0.001	TS
SR	TS vs HH_RAND	+1.32	<0.001	HH_RAND
SR	TS vs HH_GREEDY	-0.07	<0.001	TS
LR	Greedy vs GVNS	-4.38	0.048	Greedy
LR	Greedy vs HH_RAND	+3.33	<0.001	HH_RAND
LR	Greedy vs HH_GREEDY	+2.63	<0.001	HH_GREEDY
LR	TS vs GVNS	-3.96	<0.001	TS
LR	TS vs HH_RAND	+3.74	<0.001	HH_RAND
LR	TS vs HH_GREEDY	+3.04	<0.001	HH_GREEDY

core are competitive with dedicated metaheuristics on this benchmark, and that they decisively outperform the GVNS baseline.

6 Conclusions

We presented a reproducible comparison of ten methods for the k-TSP, evaluated on the 76-instance TSPLIB benchmark and the short-/long-run time protocol of Pandiri and Singh (2020), with LKH-3 as the best-known-solution reference and a Wilcoxon signed-rank analysis against the published GVNS and hyper-heuristic results. The greedy cheapest-insertion heuristic and Tabu Search are the most robust of our methods and are significantly better than GVNS under both time budgets; under the short run the greedy heuristic also matches or exceeds the HH_GREEDY hyper-heuristic. The genetic algorithm is most competitive when k is large and more time is available, while Farthest Insertion is the weakest choice for this selective problem. The proposed DCIH offers a deterministic, fully reproducible alternative of moderate quality. Future work includes hybrid metaheuristics combining the greedy construction with memory-based search, per-instance parameter tuning, dedicated node-selection rules, and an extension of the benchmark to larger instances.

References

D. L. Applegate, R. E. Bixby, V. Chvátal, and W. J. Cook. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, Princeton, NJ, 2006.

S. Arora. Polynomial time approximation schemes for euclidean traveling salesman and other geometric problems. *Journal of the ACM*, 45(5):753, 1998.

E. Balas. The prize collecting traveling salesman problem. *Networks*, 19(6):621, 1989.

T. Bektaş. The multiple traveling salesman problem: An overview of formulations and solution procedures. *Omega*, 34(3):209, 2006.

R. Bellman. Dynamic programming treatment of the travelling salesman problem. *Journal of the ACM*, 9(1):61, 1962.

J. L. Bentley. Fast algorithms for geometric traveling salesman problems. *ORSA Journal on Computing*, 4(4):387, 1992.

D. Bienstock, M. X. Goemans, D. Simchi-Levi, and D. Williamson. A note on the prize collecting traveling salesman problem. *Mathematical Programming*, 59(1):413, 1993.

M. Blanchard, A. Jacquillat, and P. Jaillet. Probabilistic bounds on k-tsp. *arXiv:2211.11063*, 2023.

C. Blum and A. Roli. Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM Computing Surveys*, 35(3):268, 2003.

N. Christofides. Worst-case analysis of a new heuristic for the travelling salesman problem, 1976.

G. A. Croes. A method for solving traveling-salesman problems. *Operations Research*, 6(6):791, 1958.

G. Dantzig, R. Fulkerson, and S. Johnson. Solution of a large-scale traveling-salesman problem. *Operations Research*, 2(4):393, 1954.

M. Dorigo and L. M. Gambardella. Ant colony system: A cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*, 1(1):53, 1997.

A. Ene, V. Nagarajan, and R. Saket. Approximation algorithms for stochastic k-tsp, 2017.

D. Feillet, P. Dejax, and M. Gendreau. Traveling salesman problems with profits. *Transportation Science*, 39(2):188, 2005.

T. A. Feo and M. G. C. Resende. Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6(2):109, 1995.

F. Glover. Future paths for integer programming and links to artificial intelligence. *Computers & Operations Research*, 13(5):533, 1986.

F. Glover and M. Laguna. *Tabu Search*. Kluwer Academic Publishers, Boston, MA, 1997.

A. Gunawan, H. C. Lau, and P. Vansteenwegen. Orienteering problem: A survey of recent variants, solution approaches and applications. *European Journal of Operational Research*, 255(2):315, 2016.

G. Gutin and A. P. Punnen, editors. *The Traveling Salesman Problem and Its Variations*. Springer, New York, 2007.

G. Gutin, A. Yeo, and A. Zverovich. Traveling salesman should not be greedy: domination analysis of greedy-type heuristics for the tsp. *Discrete Applied Mathematics*, 117(1):81, 2002.

M. Held and R. M. Karp. A dynamic programming approach to sequencing problems. *Journal of the SIAM*, 10(1):196, 1962.

K. Helsgaun. An effective implementation of the lin-kernighan traveling salesman heuristic. *European Journal of Operational Research*, 126(1):106, 2000.

K. Helsgaun. An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems, 2017.

J. H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor, MI, 1975.

S. Hougardy and X. Zhong. A survey on approximability of traveling salesman problems using the tsp-t3co definition scheme. *Annals of Operations Research*, 2025.

D. S. Johnson and L. A. McGeoch. The traveling salesman problem: A case study in local optimization. *Local Search in Combinatorial Optimization*, page 215, 1997.

R. M. Karp. Reducibility among combinatorial problems. *Complexity of Computer Computations*, page 85, 1972.

S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220 (4598):671, 1983.

J. Knox. Tabu search performance on the symmetric traveling salesman problem. *Computers & Operations Research*, 21(8):867–876, 1994.

G. Laporte. The traveling salesman problem: An overview of exact and approximate algorithms. *European Journal of Operational Research*, 59(2):231, 1992.

E. L. Lawler, J. K. Lenstra, A. H. G. Rinnooy Kan, and D. B. Shmoys. The traveling salesman problem: A guided tour of combinatorial optimization. *Wiley*, 1985.

S. Lin. Computer solutions of the traveling salesman problem. *Bell System Technical Journal*, 44 (10):2245, 1965.

O. Martin, S. W. Otto, and E. W. Felten. Large-step markov chains for the traveling salesman problem. *Complex Systems*, 5(3):299, 1991.

C. E. Miller, A. W. Tucker, and y R. A. Zemlin. Integer programming formulation of traveling salesman problems. *Journal of the ACM*, 7(4):326, 1960.

N. Mladenović and P. Hansen. Variable neighborhood search. *Computers & Operations Research*, 24(11):1097, 1997.

V. Pandiri and A. Singh. Two multi-start heuristics for the k-traveling salesman problem. *OPSEARCH*, 57:1164, 2020.

J.-Y. Potvin. Genetic algorithms for the traveling salesman problem. *Annals of Operations Research*, 63(3):337, 1996.

C. Rego, D. Gamboa, F. Glover, and C. Osterman. Traveling salesman problem heuristics: Leading methods, implementations and latest advances. *European Journal of Operational Research*, 211 (3):427, 2011.

G. Reinelt. Tsplib—a traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376, 1991.

D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis II. An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing*, 6(3):563, 1977.

E.-G. Talbi. *Metaheuristics: From Design to Implementation.* Wiley, Hoboken, NJ, 2009.

P. Vansteenwegen, W. Souffriau, and D. Van Oudheusden. The orienteering problem: A survey. *European Journal of Operational Research*, 209(1):1, 2011.

V. Černý. Thermodynamical approach to the traveling salesman problem: An efficient simulation algorithm. *Journal of Optimization Theory and Applications*, 45(1):41, 1985.

P. Venkatesh, G. Srivastava, and A. Singh. A gvns heuristic for k-tsp. *Procedia Computer Science*, 143:189, 2018.

S. Lin y B. W. Kernighan. An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21(2):498, 1973.